Introduction to Retrieval-Augmented Generation (RAG)

What is RAG?

Retrieval-Augmented Generation combines information retrieval with a language model. Instead of relying only on the model's internal knowledge, it retrieves relevant document chunks from a knowledge base and uses them to generate accurate answers.

RAG Pipeline

1. Load documents. 2. Split text into chunks. 3. Generate embeddings. 4. Store embeddings in a vector database such as FAISS. 5. Retrieve the most relevant chunks. 6. Send the retrieved context to the LLM. 7. Generate the final answer.

Document Loaders

LangChain supports PyPDFLoader, TextLoader, CSVLoader, DirectoryLoader and many others.

Text Splitting

RecursiveCharacterTextSplitter divides long documents into overlapping chunks to improve retrieval.

Embeddings

Embeddings convert text into dense vectors. Common models include sentence-transformers/all-MiniLM-L6-v2 and BAAI/bge-small-en-v1.5.

Vector Database

FAISS stores embeddings and performs similarity search using vector distance.

Retriever

A retriever finds the top-k most relevant chunks for a user query.

Prompting

The prompt includes retrieved context and the user question so the LLM answers using the provided information.

Advantages

Reduces hallucinations, supports private data, improves factual accuracy, and enables document question answering.

Mini Example

Question: What is FAISS? Answer: FAISS is a vector database library for efficient similarity search over embeddings.